

Harbinger com Ambiente Experimental



Eduardo Ogasawara

eogasawara@cefet-rj.br | <https://eic.cefet-rj.br/~eogasawara>



Workflow recorrente

01

Carregar dados.

02

Escolher ou configurar um método.

03

Executar detecção.

04

Inspecionar estrutura de saída.

05

Plotar e avaliar contra rótulos quando existirem.

Setup mínimo

Os exemplos usam uma semente comum para reprodutibilidade.

O pacote `harbinger` fornece detectores, avaliação e visualização.

`daltoolbox` aparece em exemplos que usam transformações e modelos auxiliares.

```
source(url("https://raw.githubusercontent.com/cefet-rj-dal/harbinger/master/examples/seed.R"))  
library(harbinger)  
library(daltoolbox)
```

Dados rotulados para o primeiro contato

O primeiro exemplo usa uma série pequena com rótulo de evento.

Isso permite entender entrada, saída e visualização sem discutir ainda um detector específico.

```
data(examples_anomalies)
dataset <- examples_anomalies$simple
head(dataset)
```

Visualização inicial com rótulos

Antes de detectar, visualizamos a série e os eventos conhecidos.

Contrato comum do Harbinger

`harbinger()` representa o contrato comum do pacote.

Um método recebe uma série e retorna uma saída padronizada orientada a eventos.

Esse contrato facilita trocar detectores sem trocar todo o workflow.

```
model <- harbinger()  
detection <- detect(model, dataset$serie)  
head(detection)
```

Saída padronizada

A saída contém pelo menos índice, marcação de evento e tipo. Detectores diferentes podem adicionar atributos específicos. A padronização permite avaliação e plotagem com a mesma interface.

```
idx | event | type  
1 | FALSE |  
2 | FALSE |  
...
```

Antes do método: conheça seus dados

- A escolha do detector depende da coleção: tamanho, canais, rótulos e estrutura.
- O tutorial inspeciona benchmarks antes de rodar métodos.
- Essa etapa evita escolher algoritmo apenas pelo nome.

Resumo de dataset

O helper identifica número de séries, tipo do dataset e colunas de sinal. Metadados como `idx`, `event`, `type`, `seq` e `seqlen` são separados dos sinais. Isso prepara a análise de coleções univariadas e multivariadas.

```
data(A1Benchmark)
A1Benchmark <- loadfulldata(A1Benchmark)
info <- dataset_summary(A1Benchmark)
```

har_plot() como diagnóstico

Plotar não é apenas embelezar resultado.

A visualização revela tendência, bursts, mudança de nível, eventos rotulados e ruído.

É a primeira checagem antes de ajustar janelas, limiares e detectores.

Séries univariadas e multivariadas

Univariada

Uma coluna de sinal. Entrada direta para muitos detectores. Boa para iniciar o workflow.

Multivariada

Várias colunas de sinal. Requer escolher canais ou combinar dimensões. A inspeção orienta a escolha.

Anomalia baseada em modelo

`hanr_arima()` modela o comportamento esperado da série.

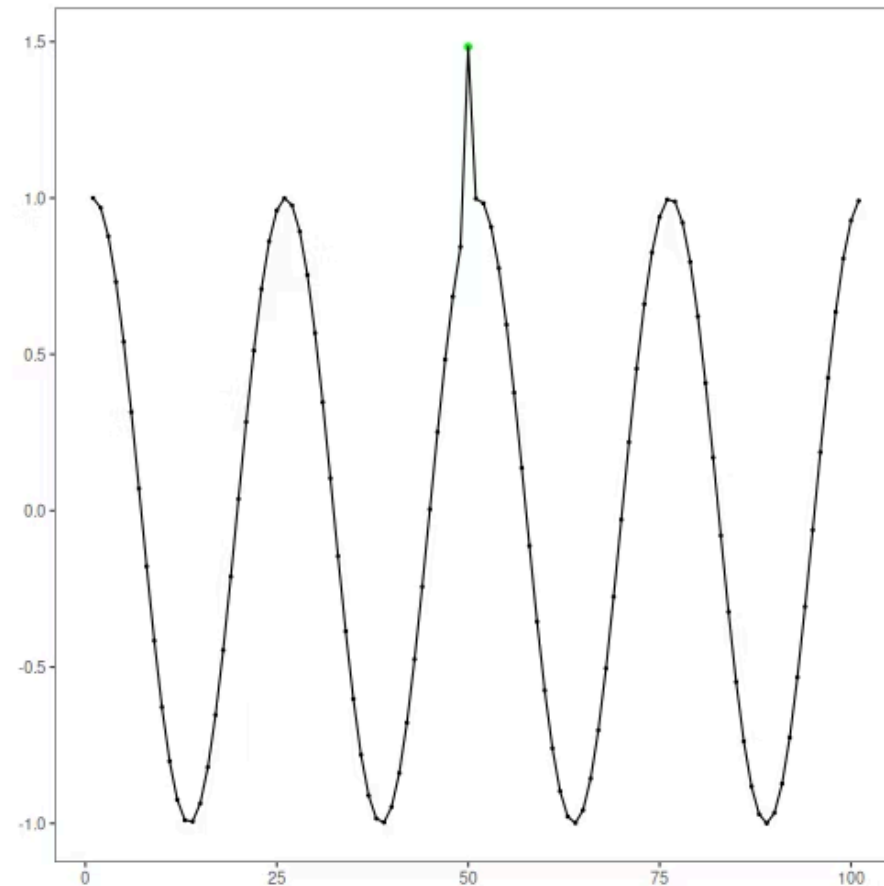
Anomalias aparecem como resíduos grandes.

Esse exemplo conecta o Harbinger à teoria de modelos temporais.

ARIMA: ajustar, detectar e avaliar

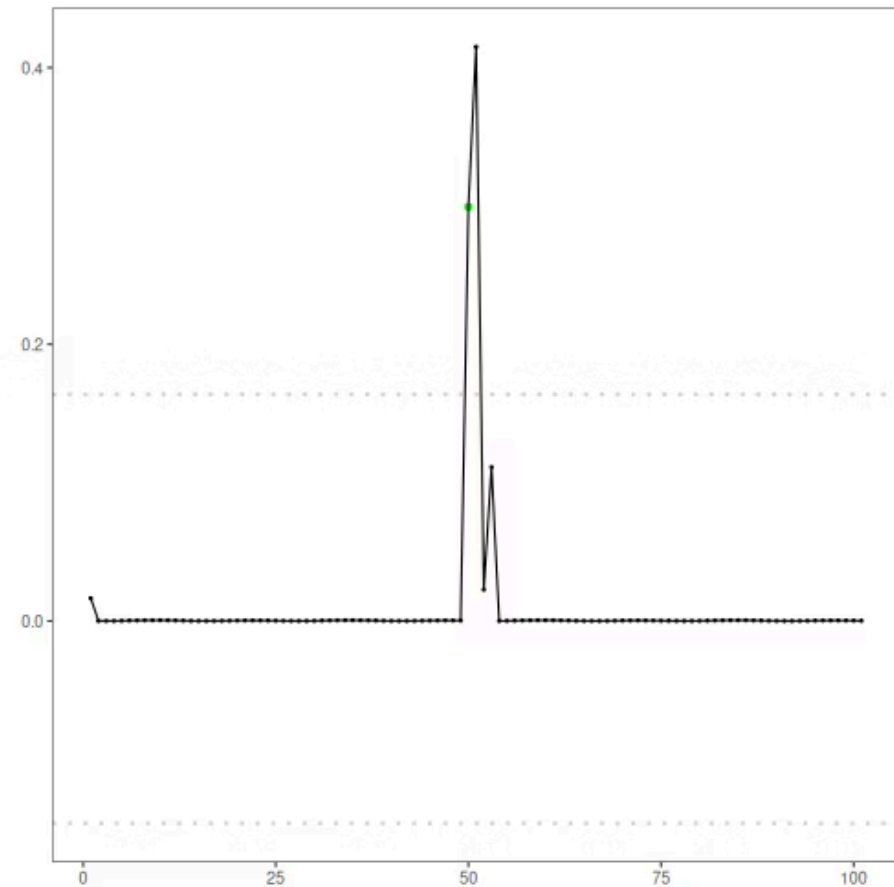
O detector é ajustado à série antes da detecção. O mesmo workflow `fit()`, `detect()` e `evaluate()` permanece.

```
model <- hanr_arima()
model <- fit(model, dataset$serie)
detection <- detect(model, dataset$serie)
evaluate(model, detection$event, dataset$event)
```



Resíduos e limiar

O resíduo explica por que o detector marcou determinados pontos.



Avaliação estrita e avaliação suave

Hard evaluation

Exige coincidência direta.

Útil para rótulos pontuais bem definidos.

```
har_eval()
```

Soft evaluation

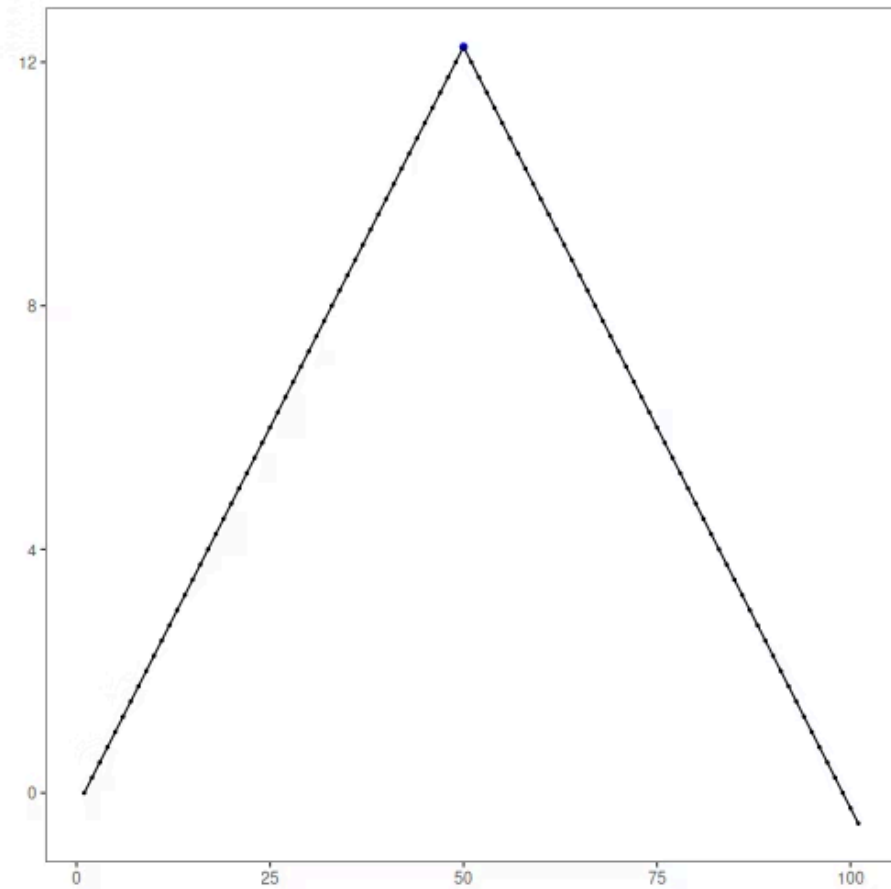
Aceita desalinhamento temporal pequeno.

Mais adequada a eventos com imprecisão temporal.

```
har_eval_soft()
```

Série com change point rotulado

A visualização mostra a série antes de aplicar AMOC.



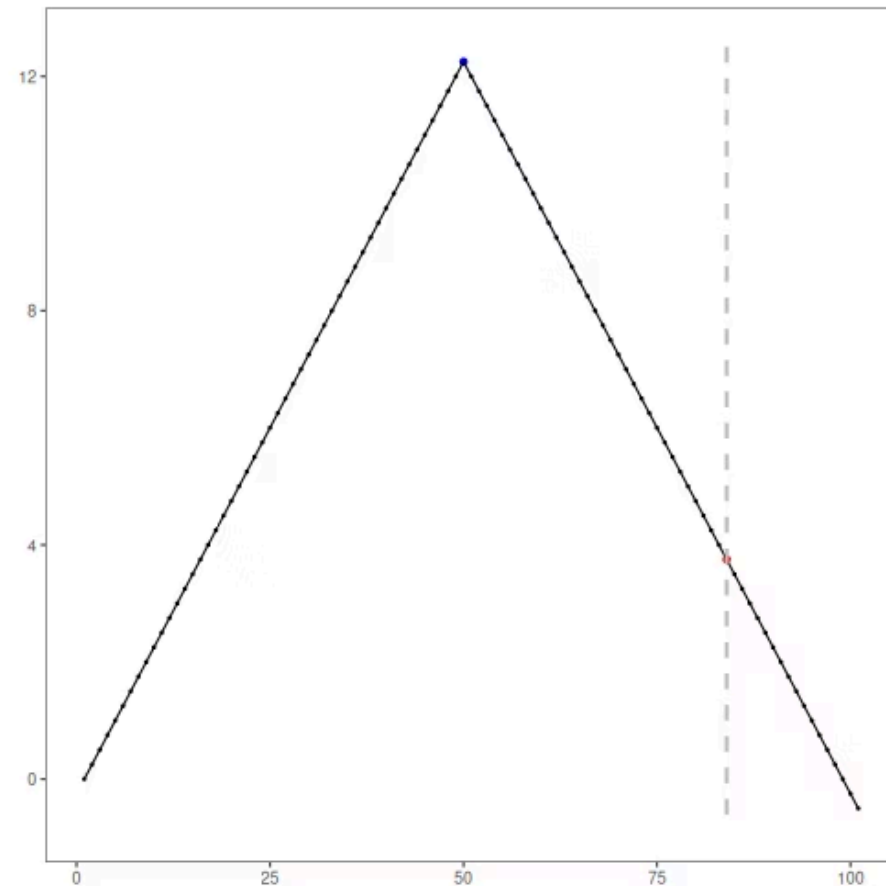
Primeiro detector de change point

- `hcp_amoc()` procura uma mudança dominante.
- O alvo não é um ponto isolado raro, mas uma transição de regime.
- É uma entrada simples antes de métodos de múltiplas mudanças.

AMOC no Harbinger

A chamada mantém a mesma estrutura dos exemplos anteriores. O resultado marca o índice detectado como `change point`.

```
model <- hcp_amoc()
detection <- detect(model, dataset$serie)
har_plot(model, dataset$serie, detection, dataset$event)
```



Transformações importam

- Antes de detectar, podemos transformar a série.
- Suavização revela padrões de baixa frequência e reduz variação local.
- A escolha da transformação muda o que o detector enxerga.

Moving average smoother

- `mas()` calcula média móvel.
- Janelas menores preservam detalhe.
- Janelas maiores produzem sinal mais suave.

```
ma_5 <- mas(dataset$serie, order = 5)  
ma_15 <- mas(dataset$serie, order = 15)
```

Série antes da discretização

O exemplo de motifs é transformado antes da análise simbólica.

SAX e XSAX no workflow

`trans_sax()` usa um alfabeto compacto. `trans_xsax()` permite uma codificação mais rica. As primeiras posições já mostram a passagem de valores para símbolos.

```
sax_model <- fit(trans_sax(alpha = 8), dataset$serie)
sax_series <- transform(sax_model, dataset$serie)
xsax_model <- fit(trans_xsax(alpha = 16), dataset$serie)
```

Motifs e discords

A trilha fecha com análise de padrões recorrentes.

Motif discovery pergunta quais subsequências se repetem.

Discords são subsequências isoladas em relação ao restante.

Detector simbólico de motifs

`hmo_sax()` conecta discretização simbólica e busca por subsequências recorrentes. Os parâmetros controlam alfabeto, tamanho da subsequência e suporte.

```
model <- hmo_sax(8, 15, 3)
model <- fit(model, dataset$serie)
detection <- detect(model, dataset$serie)
```

